04–08 – Spánek po nečinnosti

Zadání

⚠ Zadání je známé jen ze dvou vzpomínek studentů. Autor souboru píše: „Zadania máme dve. Neviem, ktoré je presnejšie, tak tu nechám obe.“

> ② Petriho sítě M/M/1, linka po přesně 10 minutách nečinnosti jde do spacího režimu, z režimu spaní ji probudí přích. požad. a trvá přesně 5 minut než se stane dostupnou.

Druhý přepis (str. 2): „M/M/1 – po 10min nečinnosti se uspí, probudí ji požadavek 5 min po svém příchodu.“

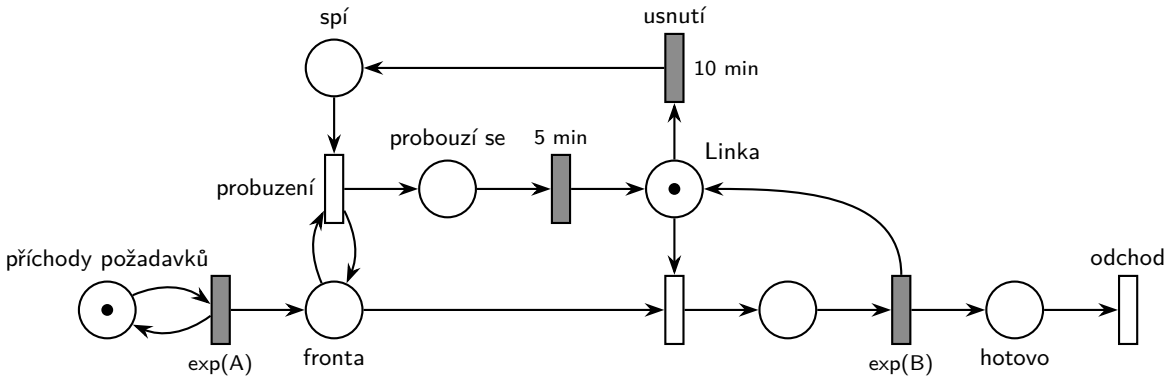
Podobné úlohy:

- 10–02 – Výpočetní služba se spánkem: stejný mechanismus (200 s a 5 s)
- 04–03 – Automat na kávu – uspání: probuzení prodlouží obsluhu
- 04–01 – Automat na kávu – údržba po nečinnosti
- 09–09 – Trafika

Interpretace

- **M/M/1:** *příchody požadavků* ⇄ `exp(A)` → *fronta* → obsazení *Linky* → `exp(B)`, které značku linky vrací → *odchod*. Časy jsou v minutách, střední doby příchodů a obsluhy zadání neuvádí, proto `exp(A)` a `exp(B)`.
- **Nečinnost** = linka nic neobsluhuje a nikdo nečeká, tedy značka je v místě *Linka*. Časovaný přechod `10 min` (*usnutí*) má jediný vstup, *Linku*, a přesouvá značku do místa *spí*. Obsazení si značku vezme a odpočet zanikne (slajd 139). Po každé obsluze začíná znovu od nuly.
- **Linka má čtyři stavy** a její značka je vždy v právě jednom: volná (*Linka*), obsluhuje, *spí*, *probouzí se*.
- **Probuzení příchozím požadavkem:** okamžitý přechod potřebuje značku *spí* a požadavek ve *frontě*. K *frontě* je připojený testovacími hranami (tam a zpět), takže požadavek ve frontě zůstane.
- **5 min, než je linka dostupná:** *probouzí se* → `5 min` → *Linka*, teprve pak požadavek linku obsadí. Linka je tedy dostupná 5 min po příchodu požadavku, který ji probudil (to říká i druhý přepis). Požadavky, které přijdou mezitím, čekají ve *frontě*.
- Doby 10 a 5 min jsou pevné.
- Požadavek po obsluze **opouští systém** přechodem *odchod*.

Petriho síť



Časté chyby

- Časovač nečinnosti na *frontě* nebo na generátoru místo na volné lince.
- Probuzení obyčejnou hranou z *fronty* bez návratu: požadavek, který linku probudil, zmizí a nikdy není obsloužen. (Pokud místo toho rovnou obsadí linku, jako v 04–03, je to v pořádku.)
- Spící linka, kterou probudí sám čas, ne příchod požadavku.
- Probouzení, které nevrací značku do *Linky* (linka zůstane nedostupná).

SIMLIB

Model je izomorfní k Petriho síti výše: každé místo, přechod a značka mají svůj protějšek v kódu.

```
// 04–08 Spánek po nečinnosti – model v SIMLIB/C++
#include "simlib.h"

const double A = 10;                    // střední interval příchodů [min] (zadání
 ↪  ho neuvádí)
const double B = 6;                     // střední doba obsluhy [min] (zadání ji
 ↪  neuvádí)

Facility Linka("Linka");                // značka linky: volná / obsluhuje / spí /
 ↪  probouzí se
Histogram Doba("Doba požadavku v systému [min]", 0, 5, 20);
int necinnosti = 0, usnuti = 0;         // počet období nečinnosti a usnutí

class Spanek;
Spanek *odpocet = nullptr;              // běžící odpočet přechodu 10 min, nebo
 ↪  žádný
Spanek *spici = nullptr;                // značka linky v místě „spí“, nebo žádná
void Volno();

class Spanek : public Process {         // značka linky na cestě usnutí → spí →
 ↪  probouzí se
  void Behavior() {
    Wait(10);                           // časovaný přechod 10 min (usnutí) doběhl
 ↪  ...
    odpocet = nullptr;
    Seize(Linka);                       // ... a vzal značku linky: místo „spí“
    usnuti++;
    spici = this;
    Passivate();                       // spí, dokud ji nevzbudí příchozí požadavek
    Wait(5);                            // místo „probouzí se“ + časovaný přechod 5
 ↪  min ...
    Release(Linka);                    // ... vrací značku do místa Linka, obsadí ji
    Volno();                            //   první požadavek z fronty
```

```
  }
};

void Volno() {                          // značka se vrátila do místa Linka
  if (!Linka.Busy()) {                  // fronta je prázdná, nikdo ji hned
   ↪ neobsadil
    necinnosti++;
    odpocet = new Spanek;               // začíná odpočet 10 min
    odpocet->Activate();
  }
}

class Pozadavek : public Process {      // transakce = značka, která prochází sítí
  void Behavior() {
    double prichod = Time;
    if (odpocet) {                      // obsazení vezme značku linky:
      odpocet->Cancel();                //   odpočet 10 min zanikne (slajd 139)
      odpocet = nullptr;
    }
    if (spici) {                        // přechod „probuzení“; požadavek zůstává
      spici->Activate();                //   ve frontě (testovací hrany)
      spici = nullptr;
    }
    Seize(Linka);                       // fronta + obsazení
    Wait(Exponential(B));               // časovaný přechod exp(B) ...
    Release(Linka);                     // ... který vrací značku linky
    Volno();
    Doba(Time - prichod);
  }                                     // konec Behavior = přechod „odchod“
};

class Generator : public Event {        // místo „příchody požadavků“ ve smyčce s
 ↪ exp(A)
  void Behavior() {
    (new Pozadavek)->Activate();        // nová značka v místě „fronta“
    Activate(Time + Exponential(A));    // další příchod
  }
};

int main() {
  Init(0, 100000);                      // modelový čas v minutách
  (new Generator)->Activate();
  Volno();                              // na začátku je linka volná: běží odpočet
 ↪ 10 min
  Run();
  Linka.Output();                       // obsazení = obsluha + spánek + probouzení
  Doba.Output();
  Print("Období nečinnosti: %d, z toho usnutí: %d\n", necinnosti, usnuti);
}
```

Petriho síť	SIMLIB
příchody požadavků ⇄ `exp(A)`	událost `Generator`, která se znovu plánuje za `Exponential(A)`
značka (požadavek)	proces `Pozadavek`
fronta + obsazení linky	`Seize(Linka)` (fronta je vnitřní fronta zařízení)
časovaný přechod `exp(B)` se sloučeným uvolněním	`Wait(Exponential(B)); Release(Linka);`
místo *Linka* (1 značka)	`Facility Linka`; značka v místě = `!Linka.Busy()`
odpočet `10 min` (*usnutí*)	proces `Spanek`, `Wait(10)`, spuštěný ve `Volno()`, kdykoli značka zůstane v místě *Linka*

Petriho síť	SIMLIB
obsazení zruší odpočet	`odpocet->Cancel()` v procesu `Pozadavek`
místo *spí*	`Spanek` drží linku (`Seize`) a je pasivní (`Passivate()`)
probuzení (testovací hrany na *frontě*)	`spici->Activate()`; požadavek pak čeká ve frontě zařízení
probouzí se → 5 `min` → *Linka*	`Wait(5); Release(Linka);` v procesu `Spanek`
přechod *odchod*	konec metody `Behavior()`

Kontrola simulací. Petriho síť: *Linka* + obsluha + *spí* + *probouzí se* = 1, linka usíná a probouzí se, všechny přechody se provádějí, fronta zůstává krátká (`make check`). SIMLIB model: v jednom běhu (100 000 min) prošlo systémem 10 048 požadavků. Z 3 414 období nečinnosti skončilo usnutím 1 204, tj. 35 %; teoreticky e^(−10/10) ≈ 37 %, protože doba do dalšího příchodu je exponenciální se středem A = 10 min. Zařízení `Linka` bylo obsazené 79 % času, protože ho drží i spící linka: 60 % obsluha (B/A), 6 % probouzení (1 204 × 5 / 100 000) a zbytek spánek (`make simlib`).

Původ a zdroje

Původ: zkouška 2022/23, 2. termín (v hlavičce přepisu „IMS 1.op. F: 2023“), skupina F, otázka 2, 10 bodů.

Původní znění:

> ② Petriho sítě M/M/1, linka po 10m nečinnosti jde do spacího režimu, z režimu spaní ji probudí přích. požad. a trvá 5 min než se stane dostupnou.

Zdroje:

- `Fitcheats/Zkoušky z minulých let/2022-23 - 2. termín F.pdf`, str. 1 a 2 (dva ručně psané přepisy zadání od různých studentů)

Existující řešení:

- Ve zdrojích není žádné řešení. Stejná síť (jen s jinými časy) je v řešení půlsemestrálky 10–02 – Výpočetní služba se spánkem.